Imam Abdulrahman Bin Faisal University
College of Computer Science and Information Technology

ARTI 404 — Natural Language Processing

Lab 6 — Deep Learning for NLP

Student Name	Ahmed Albuainain
Student ID	2240006128
Section	7MA1

Deep Learning for NLP

Deep Learning is a type of Machine Learning that uses **Neural Networks with multiple layers** to learn patterns from data.

Deep Learning can be used in NLP tasks such as:
- Text Classification
- Sentiment Analysis
- Question Answering

In this lab, we will build a **deep learning model** to classify Yelp restaurant reviews as:

- **0 = Negative**
- **1 = Positive**

Setup

Run this cell first. It installs the libraries used in this lab and downloads the NLTK tokenizer data.

```
# Install the libraries used in this lab (safe to re-run)
import sys
!{sys.executable} -m pip install torch gensim

import nltk
nltk.download('punkt')
nltk.download('punkt_tab')

print("Setup complete.")
```

```
Requirement already satisfied: torch in c:\users\ahmad\miniconda3\lib\site-packages (2.14.1)
Requirement already satisfied: gensim in c:\users\ahmad\miniconda3\lib\site-packages (4.4.0)
Requirement already satisfied: filelock in c:\users\ahmad\miniconda3\lib\site-packages (from
torch) (4.1.1)
Requirement already satisfied: typing-extensions>=4.10.0 in
c:\users\ahmad\miniconda3\lib\site-packages (from torch) (4.12.2)
Requirement already satisfied: setuptools>=77.0.3 in c:\users\ahmad\miniconda3\lib\site-
packages (from torch) (78.1.1)
Requirement already satisfied: sympy>=1.13.3 in c:\users\ahmad\miniconda3\lib\site-packages
(from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in c:\users\ahmad\miniconda3\lib\site-packages
(from torch) (3.6.1)
Requirement already satisfied: jinja2 in c:\users\ahmad\miniconda3\lib\site-packages (from
torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in c:\users\ahmad\miniconda3\lib\site-packages
(from torch) (2026.9.0)
Requirement already satisfied: numpy>=1.18.5 in c:\users\ahmad\miniconda3\lib\site-packages
(from gensim) (2.3.5)

[... installation log truncated ...]
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\ahmad\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\ahmad\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```





1. Load the Yelp Dataset

Each row contains:

- a restaurant review
- its sentiment label

```
import pandas as pd

df = pd.read_csv("07-yelp-dataset.txt", sep="\t", names=["review", "label"])

print("Number of reviews:", len(df))
df.head()
```

```
Number of reviews: 1000
```

	review	label
0	Wow... Loved this place.	1
1	Crust is not good.	0
2	Not tasty and the texture was just nasty.	0
3	Stopped by during the late May bank holiday of...	1
4	The selection on the menu was great and so wer...	1

```
print(df["label"].value_counts())
```

```
label
1    500
0    500
Name: count, dtype: int64
```

2. Preprocessing the Dataset

```python
import re

def preprocess_text(text):

    # 1. Convert to lowercase
    text = text.lower()

    # 2. Remove punctuation and special characters
    text = re.sub(r'[^a-zA-Z\s]', '', text)

    # 3. Remove extra spaces
    text = re.sub(r'\s+', ' ', text).strip()

    return text

df["review"] = df["review"].apply(preprocess_text)
df.head()
```

	review	label
0	wow loved this place	1
1	crust is not good	0
2	not tasty and the texture was just nasty	0
3	stopped by during the late may bank holiday of...	1
4	the selection on the menu was great and so wer...	1

3. Split the Data

We divide the dataset into:

- **80% training data** — used to train the model
- **20% testing data** — used to evaluate the model

`stratify=y` keeps the proportion of positive and negative reviews similar in both sets.

```python
from sklearn.model_selection import train_test_split

X = df["review"]
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
random_state=42)

print("Training reviews:", len(X_train))
print("Testing reviews:", len(X_test))
```

```
Training reviews: 800
Testing reviews: 200
```

4. Convert Text to TF-IDF Vectors

A neural network cannot directly understand text.

TF-IDF converts each review into a vector of numbers.

```python
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(max_features=1000, lowercase=True)

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print("Training TF-IDF shape:", X_train_tfidf.shape)
print("Testing TF-IDF shape:", X_test_tfidf.shape)
```

```
Training TF-IDF shape: (800, 1000)
Testing TF-IDF shape: (200, 1000)
```

5. Convert the Data to PyTorch Tensors

PyTorch neural networks work with **tensors**.

```python
import torch
import torch.nn as nn

# Convert TF-IDF data to PyTorch tensors
X_train_tensor = torch.FloatTensor(X_train_tfidf.toarray())
X_test_tensor = torch.FloatTensor(X_test_tfidf.toarray())

y_train_tensor = torch.FloatTensor(y_train.values).reshape(-1, 1)
```

6. Build a Simple Neural Network

Our network has:

1. **Input layer** — receives the TF-IDF features
2. **Hidden layer** — learns useful patterns
3. **ReLU activation** — adds non-linearity
4. **Output layer** — produces one value for binary classification

We do **not** put `Sigmoid` inside the model because `BCEWithLogitsLoss` applies it internally in a numerically stable way.

```
input_size = X_train_tensor.shape[1]
# torch.size([800, 1000])

model = nn.Sequential(
    nn.Linear(input_size, 64),   # Input layer
    nn.ReLU(),

    nn.Linear(64, 32),           # Hidden layer
    nn.ReLU(),

    nn.Linear(32, 1)             # Output layer
)

print(model)
```

```
Sequential(
  (0): Linear(in_features=1000, out_features=64, bias=True)
  (1): ReLU()
  (2): Linear(in_features=64, out_features=32, bias=True)
  (3): ReLU()
  (4): Linear(in_features=32, out_features=1, bias=True)
)
```

7. Choose the Loss Function and Optimizer

- **BCEWithLogitsLoss** is used for binary classification.
  To measure the model's mistakes.
- **Adam** is responsible for changing the model's weights so that the loss becomes smaller.

```
loss_function = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

8. Train the Neural Network

One **epoch** means the model has seen the full training dataset once.

For every epoch:

1. Make predictions
2. Calculate the loss
3. Calculate gradients with backpropagation
4. Update the weights

```python
epochs = 10

for epoch in range(epochs):
    model.train()

    # 1. Forward pass
    outputs = model(X_train_tensor)

    # 2. Calculate loss
    loss = loss_function(outputs, y_train_tensor)

    # 3. Backpropagation
    loss.backward()

    # 4. Update model weights
    optimizer.step()

    if (epoch + 1) % 5 == 0:
        print(f"Epoch {epoch + 1}/{epochs} - Loss: {loss.item():.4f}")
```

```
Epoch 5/10 - Loss: 0.6566
Epoch 10/10 - Loss: 0.4791
```

9. Evaluate the Model

The model outputs raw scores called **logits**.

We apply `sigmoid` to convert them to probabilities between 0 and 1.

- Probability **≥ 0.5 → Positive (1)**
- Probability **< 0.5 → Negative (0)**

```python
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

model.eval()

with torch.no_grad():
    test = model(X_test_tensor)
    test_probabilities = torch.sigmoid(test)
    predictions = (test_probabilities >= 0.5).int().numpy().flatten()

accuracy = accuracy_score(y_test, predictions)

print(f"Test Accuracy: {accuracy:.3f}")
```

```
Test Accuracy: 0.735
```

10. Classification Report and Confusion Matrix

The classification report shows:

- **Precision**
- **Recall**
- **F1-score**

The confusion matrix shows how many reviews were classified correctly and incorrectly.

```
print("Classification Report:")
print(classification_report(y_test, predictions, target_names=["Negative", "Positive"]))

print("Confusion Matrix:")
print(confusion_matrix(y_test, predictions))
```

```
Classification Report:
              precision    recall  f1-score   support

    Negative       0.73      0.72      0.72        96
    Positive       0.74      0.75      0.75       104

    accuracy                           0.73       200
   macro avg       0.73      0.73      0.73       200
weighted avg       0.73      0.73      0.73       200

Confusion Matrix:
[[69 27]
 [26 78]]
```

Task 1 – Represent Reviews Using Word2Vec

In the previous part of the lab, TF-IDF was used to convert each review into numerical features.
Now, use Word2Vec instead.

Requirements:
Tokenize the training and testing reviews.
Train a Word2Vec model using the training reviews.
Use:
vector_size = 200
window = 5
min_count = 1
sg = 1

Convert each review into a single vector.
Convert the resulting vectors into PyTorch tensors.

```python
# Task 1: Represent the reviews using Word2Vec instead of TF-IDF
import numpy as np
import torch
import gensim
from nltk.tokenize import word_tokenize

# 1. Tokenize the training and the testing reviews
train_tokens = [word_tokenize(review) for review in X_train]
test_tokens = [word_tokenize(review) for review in X_test]

print("Tokenized training reviews:", len(train_tokens))
print("Tokenized testing reviews :", len(test_tokens))
print("Example of a tokenized review:", train_tokens[0][:12])

# 2. Train a Word2Vec model on the TRAINING reviews only,
#    so that no information from the test set leaks into the model
w2v_model = gensim.models.Word2Vec(train_tokens,
                                   vector_size=200,
                                   window=5,
                                   min_count=1,
                                   sg=1)

print("\nVocabulary size:", len(w2v_model.wv))


# 3. Convert each review into a single vector by averaging its word vectors
def review_to_vector(tokens, model):
    vectors = [model.wv[word] for word in tokens if word in model.wv]
    if len(vectors) == 0:              # a review with no known word
        return np.zeros(model.vector_size)
    return np.mean(vectors, axis=0)


X_train_w2v = np.array([review_to_vector(t, w2v_model) for t in train_tokens])
X_test_w2v = np.array([review_to_vector(t, w2v_model) for t in test_tokens])

print("Training matrix shape:", X_train_w2v.shape)
print("Testing matrix shape :", X_test_w2v.shape)

# 4. Convert the resulting vectors into PyTorch tensors
X_train_w2v_tensor = torch.FloatTensor(X_train_w2v)
X_test_w2v_tensor = torch.FloatTensor(X_test_w2v)
y_train_w2v_tensor = torch.FloatTensor(y_train.values).reshape(-1, 1)

print("\nTensor shapes:")
print("  X_train:", tuple(X_train_w2v_tensor.shape))
print("  X_test :", tuple(X_test_w2v_tensor.shape))
print("  y_train:", tuple(y_train_w2v_tensor.shape))
```

```
Tokenized training reviews: 800
Tokenized testing reviews : 200
Example of a tokenized review: ['the', 'worst', 'was', 'the', 'salmon', 'sashimi']

Vocabulary size: 1800
Training matrix shape: (800, 200)
Testing matrix shape : (200, 200)

Tensor shapes:
  X_train: (800, 200)
  X_test : (200, 200)
  y_train: (800, 1)
```

Task 2 – Build a Different Neural Network

Use the Word2Vec vectors from Task 1 as the input to a new neural network.
Your network must contain:
An input layer
Three hidden layers
ReLU activation functions
One output neuron
Use the following architecture:

Input

↓

128 neurons

↓

64 neurons

↓

32 neurons

↓

1 output neuron

Train the model using:
, BCEWithLogitsLoss
, Adam optimizer
, Learning rate = 0.001
, 15 epochs.
Then calculate the test accuracy.

```python
# Task 2: Build a deeper neural network on top of the Word2Vec vectors
import torch
import torch.nn as nn

torch.manual_seed(42)

input_size_w2v = X_train_w2v_tensor.shape[1]   # 200 features from Word2Vec

model2 = nn.Sequential(
    nn.Linear(input_size_w2v, 128),   # Input layer
    nn.ReLU(),

    nn.Linear(128, 64),               # Hidden layer 1
    nn.ReLU(),

    nn.Linear(64, 32),                # Hidden layer 2
    nn.ReLU(),

    nn.Linear(32, 1)                  # Output neuron
)

print(model2)
```

```
Sequential(
  (0): Linear(in_features=200, out_features=128, bias=True)
  (1): ReLU()
  (2): Linear(in_features=128, out_features=64, bias=True)
  (3): ReLU()
  (4): Linear(in_features=64, out_features=32, bias=True)
  (5): ReLU()
  (6): Linear(in_features=32, out_features=1, bias=True)
)
```

```python
# Loss function, optimizer and training loop
loss_function2 = nn.BCEWithLogitsLoss()
optimizer2 = torch.optim.Adam(model2.parameters(), lr=0.001)

epochs = 15

for epoch in range(epochs):
    model2.train()

    optimizer2.zero_grad()                # clear the gradients of the previous step

    outputs = model2(X_train_w2v_tensor)                  # 1. Forward pass
    loss = loss_function2(outputs, y_train_w2v_tensor)    # 2. Calculate the loss
    loss.backward()                                       # 3. Backpropagation
    optimizer2.step()                                     # 4. Update the weights

    print(f"Epoch {epoch + 1}/{epochs} - Loss: {loss.item():.4f}")
```

```
Epoch 1/15 - Loss: 0.6964
Epoch 2/15 - Loss: 0.6959
Epoch 3/15 - Loss: 0.6955
Epoch 4/15 - Loss: 0.6951
Epoch 5/15 - Loss: 0.6948
Epoch 6/15 - Loss: 0.6945
Epoch 7/15 - Loss: 0.6942
Epoch 8/15 - Loss: 0.6939
Epoch 9/15 - Loss: 0.6937
Epoch 10/15 - Loss: 0.6935
Epoch 11/15 - Loss: 0.6934
Epoch 12/15 - Loss: 0.6933
Epoch 13/15 - Loss: 0.6933
Epoch 14/15 - Loss: 0.6933
Epoch 15/15 - Loss: 0.6933
```

```python
# Calculate the test accuracy of the Word2Vec model
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

model2.eval()

with torch.no_grad():
    test_logits = model2(X_test_w2v_tensor)
    test_probabilities = torch.sigmoid(test_logits)
    predictions_w2v = (test_probabilities >= 0.5).int().numpy().flatten()

accuracy_w2v = accuracy_score(y_test, predictions_w2v)

print(f"Test Accuracy  (Word2Vec + 3 hidden layers) : {accuracy_w2v:.3f}")
print(f"Test Accuracy  (TF-IDF  + 1 hidden layer)   : {accuracy:.3f}")

print("\nClassification Report:")
print(classification_report(y_test, predictions_w2v,
                            target_names=["Negative", "Positive"]))

print("Confusion Matrix:")
print(confusion_matrix(y_test, predictions_w2v))
```

```
Test Accuracy  (Word2Vec + 3 hidden layers) : 0.480
Test Accuracy  (TF-IDF  + 1 hidden layer)   : 0.735

Classification Report:
              precision    recall  f1-score   support

    Negative       0.48      1.00      0.65        96
    Positive       0.00      0.00      0.00       104

    accuracy                           0.48       200
   macro avg       0.24      0.50      0.32       200
weighted avg       0.23      0.48      0.31       200

Confusion Matrix:
[[ 96   0]
 [104   0]]
```

```
c:\Users\ahmad\miniconda3\Lib\site-packages\sklearn\metrics\_classification.py:1833:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\ahmad\miniconda3\Lib\site-packages\sklearn\metrics\_classification.py:1833:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\ahmad\miniconda3\Lib\site-packages\sklearn\metrics\_classification.py:1833:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

Summary of the Tasks

Baseline (TF-IDF + one hidden layer). The Yelp dataset contains 1,000 restaurant reviews, perfectly balanced at 500 positive and 500 negative. After preprocessing (lower casing, removing punctuation and extra spaces) the reviews were split 80/20 into 800 training and 200 testing examples, converted into 1,000 TF-IDF features and fed into a small network (1000 → 64 → 32 → 1) trained for 10 epochs with Adam at a learning rate of 0.01. The test accuracy was **0.735**, with precision and recall close to 0.73 on both classes.

Task 1 — Word2Vec representation. The training and testing reviews were tokenized, and a Skip-gram Word2Vec model (`vector_size=200, window=5, min_count=1, sg=1`) was trained on the **training reviews only**, so that no information from the test set could leak into the representation. The resulting vocabulary contains **1,800** words. Each review was then turned into a single 200-dimensional vector by averaging the vectors of the words it contains, producing a 800 × 200 training matrix and a 200 × 200 testing matrix, which were converted into PyTorch tensors.

Task 2 — Deeper network. A four-layer network was built exactly as required (200 → 128 → 64 → 32 → 1 with ReLU between the layers) and trained with `BCEWithLogitsLoss`, the Adam optimizer, a learning rate of 0.001 and 15 epochs.

Model	Representation	Test accuracy
1 hidden layer, 10 epochs, lr = 0.01	TF-IDF (1000 features)	**0.735**
3 hidden layers, 15 epochs, lr = 0.001	Word2Vec (200 features)	**0.480**

Discussion of the result. The deeper Word2Vec model scored lower than the simple TF-IDF baseline, and the confusion matrix shows why: it predicted *Negative* for all 200 test reviews (96 correct, 104 wrong), which is the behaviour of a model that has not actually learned anything yet. The training loss confirms this — it moved only from 0.6964 to 0.6933 across the fifteen epochs, and ln(2) ≈ 0.693 is exactly the loss of a model that outputs 50/50 for every example.

The reason is the training budget rather than the representation. The loop performs full-batch gradient descent, so fifteen epochs means only **fifteen weight updates in total**, and at a learning rate of 0.001 those fifteen steps are far too small to move a network of this size away from its random initialisation. The baseline, by contrast, used a learning rate ten times larger. Training the same architecture in mini-batches, or for several hundred epochs, or at a higher learning rate would let the Word2Vec model reach a comparable accuracy; the experiment as specified simply stops before learning begins.

A second, smaller factor is that averaging the word vectors of a review discards word order, and the Word2Vec model itself was trained on only 800 short reviews, which is a very small corpus for learning useful embeddings.